

JUNE 2020

STUDENT RECORD MANAGEMENT SYSTEM REPORT

Course: IT 214
Section :4858

RAND ALSUHAIBANI 451203409

AMAL ALHARBI 432206922

REEM MOHAMMED 451203239

RANEEM MOHSEN 431201775

SYSTEM DESIGN AND ARCHITECTURE:

The Student Record Management System (SRMS) is a system used to organize and manage student information. It allows the user to add, update, delete, save, and load student records using files.

The system is built using object-oriented programming. Each part of the system is represented as a class, which helps make the code easier to understand and organize.

The system is divided into simple sections. One section is responsible for handling student data and files. Another section manages the main operations such as updating records and generating reports. The user uses the system through a simple menu.

The system also uses multithreading to perform more than one task at the same time. For example, the system can save data while another task is running. This helps the system work more smoothly.

In general, the system design is simple and clear, and it can be improved or extended later if needed.

CLASSES, METHODS, AND FUNCTIONS

PERSON CLASS:

The Person class represents a general person in the system and is mainly used as a base class. It stores the name attribute and provides the `getName()` method to retrieve the name and the `setName()` method to update it. This class helps apply the inheritance concept, as it allows other classes to reuse common attributes instead of repeating them.

COURSE CLASS:

The Course class represents a university course in the system. It stores the course code, course name, and credit hours. The class provides methods such as `getCourseCode()`, `getCourseName()`, and `getCreditHours()` to access course information. This class helps organize course-related data and allows students to be associated with multiple courses

STUDENT CLASS:

The Student class represents a student and extends the Person class. It stores important student information such as ID, GPA, department, and academic year. The class includes getter methods like `getId()`, `getGpa()`, `getDepartment()`, and `getYear()` to access student data, and setter methods such as `setGpa()`, `setDepartment()`, and `setYear()` to update this information. In addition, the Student class manages the student's courses using the `addCourse()` and `removeCourse()` methods. The `toString()` method is overridden to return student data in CSV format, which is useful when saving data to a file.

STUDENTMANAGER CLASS:

The Course class represents a university course in the system. It stores the course code, course name, and credit hours. The class provides methods such as `getCourseCode()`, `getCourseName()`, and `getCreditHours()` to access course information. This class helps organize course-related data and allows students to be associated with multiple courses

CLASSES, METHODS, AND FUNCTIONS

FILEHANDLER CLASS:

The FileHandler class handles reading and writing student data to files. It includes the `saveToFile()` method, which saves student records into a CSV file, and the `loadFromFile()` method, which reads the data from the file and recreates student objects. This class uses exception handling to manage file-related errors, ensuring the program does not crash when files are missing or contain incorrect data.

MAINTEST CLASS :

The MainTest class is used to run and test the system. It contains the `main()` method, which creates sample student data, saves it to a file, loads it again, and generates reports. This class is mainly used to verify that all parts of the system work correctly together.

REPORTGENERATOR CLASS :

The ReportGenerator class is responsible for generating different types of reports based on student data. It provides the `reportByGPA()` method to display students sorted by GPA, the `reportByDepartment()` method to group students by department, and the `reportByYear()` method to group students by academic year. These reports help present the data in a clear and organized way.

OOP PRINCIPLES , EXCEPTION HANDLING

Inheritance

Inheritance is applied by making the Student class inherit from the Person class. This promotes code reuse and represents real-world relationships.

Encapsulation

All sensitive attributes are declared as private and accessed through getter and setter methods. This ensures controlled access to student data.

Composition

Composition is used where:

- A Student contains a list of Course objects.
- StudentManager contains a list of Student objects.

Polymorphism

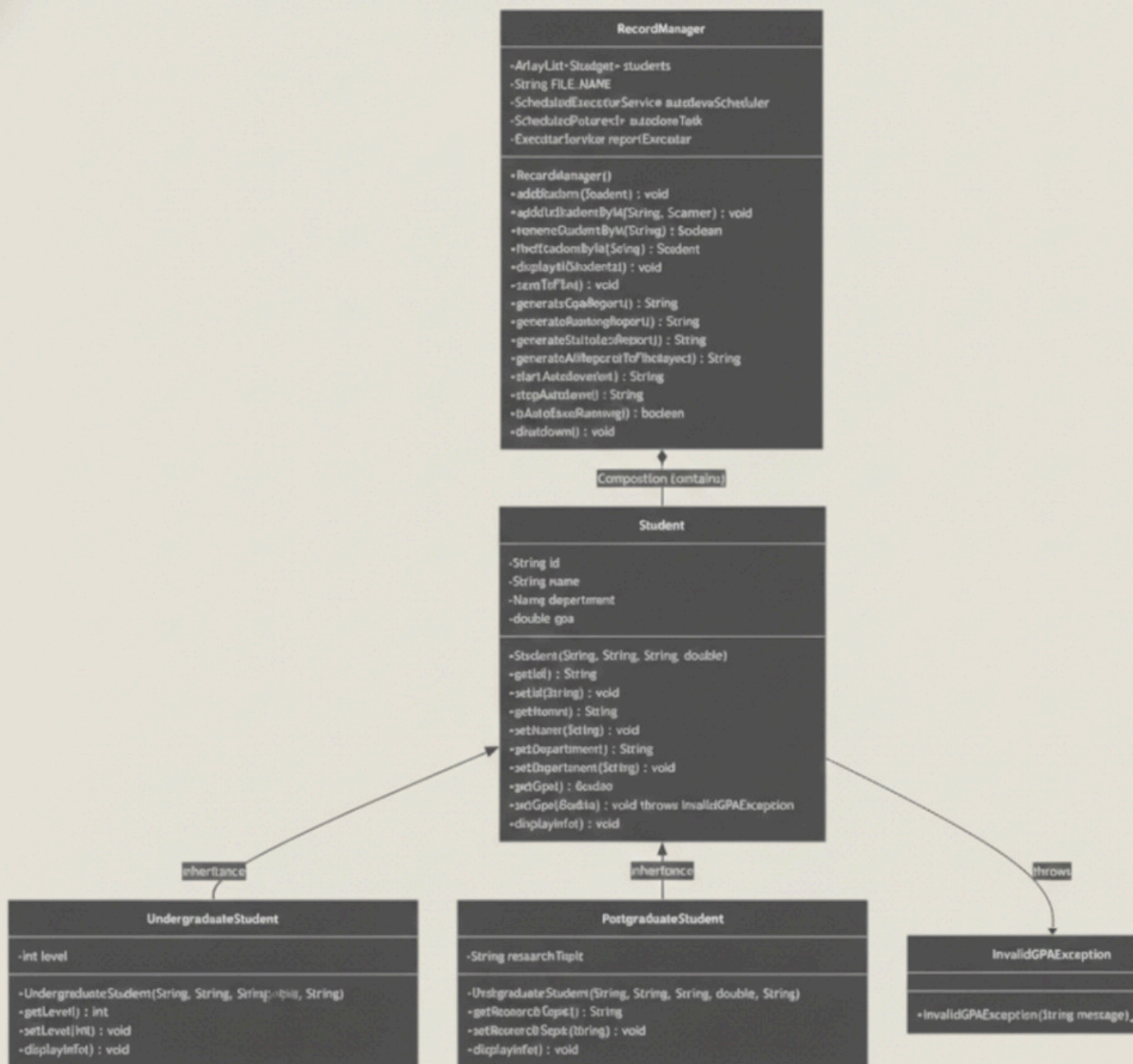
Polymorphism is demonstrated through method overriding, particularly in multithreading where different threads override the run() method to perform different tasks.

Exception Handling

Exception handling is implemented to manage invalid input, file-related errors, and runtime exceptions. This prevents system crashes and protects sensitive data from being exposed.

04

UML



CHALLENGES & SOLUTIONS

1. Challenge: Managing large student data files

- Description: Handling and storing a growing number of student records efficiently, especially when performing operations like search, update, and delete.

- Solution: Implemented file handling with structured data storage and efficient search algorithms to ensure fast access and modification of records.

2. Challenge: Ensuring data consistency

- Description: Maintaining accurate and consistent student information across multiple operations and preventing data corruption.

- Solution: Used proper validation checks before adding or updating records, and ensured that all file operations are done safely with error handling.

3. Challenge: User-friendly interface

- Description: Making the system easy to use for both students and administrators without requiring technical knowledge.

- Solution: Designed a simple console-based interface with clear prompts, menus, and messages to guide the user through all operations.

4. Challenge: Generating reports based on multiple criteria

- Description: Creating detailed reports like GPA ranking, department-wise lists, or year-wise student lists.

- Solution: Implemented sorting and filtering mechanisms that allow the system to generate accurate and customized reports efficiently

GITHUB COLLABORATION

GitHub was used to facilitate collaboration among team members. Each member could work on their tasks independently while keeping track of changes, resolving conflicts, and merging updates efficiently. This ensured smooth teamwork, version control, and proper management of the project files

FUTURE IMPROVEMENTS

The Student Record Management System can be improved in the future by:

- Adding a graphical user interface (GUI) for easier interaction.
- Supporting web and mobile access for convenience.
- Providing more detailed reports and analytics.
- Integrating notifications for important updates.
- Enhancing search speed and handling of large datasets

LEGAL & ETHICAL CONSIDERATIONS

When developing the Student Record Management System, several legal and ethical considerations were taken into account:

- Legal:
 - Protecting student data privacy according to local and international regulations.
 - Storing data securely to prevent unauthorized access.
 - Complying with intellectual property rights for any code or tools used.
- Ethical:
 - Handling student information confidentially and not sharing it with unauthorized parties.
 - Ensuring data accuracy and preventing any manipulation.
 - Designing the system fairly without discrimination between students or departments